



Data Structures And Algorithms

Name : Samreen Zia
Submitted to : Mr. Rehan Ahmed
Semester : BSE (3A)
Date : 03-Jan- 2026

LAB NO 14

Title: Project submission

Project Title

Student Record Management System using Binary Search Tree (BST)

Problem Statement

Managing student records efficiently is a common problem in educational institutions. Traditional methods such as arrays or lists become inefficient when the number of students grows, especially for searching, inserting, or deleting records.

Problem

- Design and implement a Student Record Management System that:
- Stores student records
- Allows insertion of new students
- Searches a student by roll number
- Deletes a student record
- Displays all student records in sorted order

Data Structure Used:

Binary Search Tree (BST)

Why BST?

- Fast searching, insertion, and deletion: $O(\log n)$ (average case)
- Maintains data in sorted order
- Efficient for dynamic data
- Purpose of BST in this Project

- Each node stores a student record
- Roll number is used as the key
- Left subtree contains smaller roll numbers
- Right subtree contains larger roll numbers

Program:

```
#include <iostream>
using namespace std;

// Structure for student node
struct Node {
    int rollNo;
    string name;
    Node* left;
    Node* right;
};

// Create new node
Node* createNode(int roll, string name) {
    Node* newNode = new Node();
    newNode->rollNo = roll;
    newNode->name = name;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

// Insert a student record into BST
Node* insert(Node* root, int roll, string name) {
    if (root == NULL)
        return createNode(roll, name);

    if (roll < root->rollNo)
        root->left = insert(root->left, roll, name);
    else if (roll > root->rollNo)
        root->right = insert(root->right, roll, name);

    return root;
}

// Search student by roll number
Node* search(Node* root, int roll) {
    if (root == NULL || root->rollNo == roll)
```

```

        return root;

    if (roll < root->rollNo)
        return search(root->left, roll);

    return search(root->right, roll);
}

// Find minimum value node
Node* findMin(Node* root) {
    while (root->left != NULL)
        root = root->left;
    return root;
}

// Delete student record
Node* deleteNode(Node* root, int roll) {
    if (root == NULL)
        return root;

    if (roll < root->rollNo)
        root->left = deleteNode(root->left, roll);
    else if (roll > root->rollNo)
        root->right = deleteNode(root->right, roll);
    else {
        // Node with one or no child
        if (root->left == NULL) {
            Node* temp = root->right;
            delete root;
            return temp;
        }
        else if (root->right == NULL) {
            Node* temp = root->left;
            delete root;
            return temp;
        }
        // Node with two children
        Node* temp = findMin(root->right);
        root->rollNo = temp->rollNo;
        root->name = temp->name;
        root->right = deleteNode(root->right, temp->rollNo);
    }
    return root;
}

```

```

// Display records in sorted order
void inorder(Node* root) {
    if (root != NULL) {
        inorder(root->left);
        cout << "Roll No: " << root->rollNo
            << ", Name: " << root->name << endl;
        inorder(root->right);
    }
}

// Main function
int main() {
    Node* root = NULL;
    int choice, roll;
    string name;

    do {
        cout << "\n--- Student Record Management System ---\n";
        cout << "1. Insert Student\n";
        cout << "2. Search Student\n";
        cout << "3. Delete Student\n";
        cout << "4. Display All Students\n";
        cout << "5. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                cout << "Enter Roll Number: ";
                cin >> roll;
                cout << "Enter Name: ";
                cin >> name;
                root = insert(root, roll, name);
                break;

            case 2:
                cout << "Enter Roll Number to Search: ";
                cin >> roll;
                if (search(root, roll))
                    cout << "Student Found!\n";
                else
                    cout << "Student Not Found!\n";
                break;

            case 3:

```

```

        cout << "Enter Roll Number to Delete: ";
        cin >> roll;
        root = deleteNode(root, roll);
        cout << "Record Deleted (if existed).\n";
        break;

    case 4:
        cout << "\nStudent Records:\n";
        inorder(root);
        break;
    }
} while (choice != 5);

return 0;
}

```

Output:

```

--- Student Record Management System ---
1. Insert Student
2. Search Student
3. Delete Student
4. Display All Students
5. Exit
Enter choice: 1
Enter Roll Number: 101
Enter Name: John

--- Student Record Management System ---
1. Insert Student
2. Search Student
3. Delete Student
4. Display All Students
5. Exit
Enter choice: 1
Enter Roll Number: 102
Enter Name: Alice

--- Student Record Management System ---
1. Insert Student
2. Search Student
3. Delete Student
4. Display All Students
5. Exit
Enter choice: 1
Enter Roll Number: 103
Enter Name: Bob

--- Student Record Management System ---
1. Insert Student
2. Search Student
3. Delete Student
4. Display All Students
5. Exit
Enter choice: 4

```

```
Student Records:
Roll No: 101, Name: John
Roll No: 102, Name: Alice
Roll No: 103, Name: Bob
```

```
--- Student Record Management System ---
1. Insert Student
2. Search Student
3. Delete Student
4. Display All Students
5. Exit
Enter choice: 2
Enter Roll Number to Search: 102
Student Found!
```

```
--- Student Record Management System ---
1. Insert Student
2. Search Student
3. Delete Student
4. Display All Students
5. Exit
Enter choice: 3
Enter Roll Number to Delete: 102
Record Deleted (if existed).
```

```
--- Student Record Management System ---
1. Insert Student
2. Search Student
3. Delete Student
4. Display All Students
5. Exit
Enter choice: 4
```

```
Student Records:
Roll No: 101, Name: John
Roll No: 103, Name: Bob
```

```
--- Student Record Management System ---
1. Insert Student
2. Search Student
3. Delete Student
4. Display All Students
5. Exit
Enter choice: 5
```